PROJEKTDOKUMENTATION

Smart Environment Monitor – Strandvattenkvalitet

Författare: Komlan
Datum: 2026-04-13
Version: 1.0
Status: Produktionsredo

Innehåll

1. Inledning 3

2. Arkitektur 3

3. Teknikval 4

4. Kravuppfyllelse 4

5. Instruktioner för körning 5

5.1 Produktion (Docker Compose) – rekommenderas 5

5.2 Lokal utveckling (utan Docker) 5

5.3 Övervakning och driftlarm 5

6. Säkerhet och åtkomstkontroll 6

6.1 Enhetsautentisering (IoT) och rate limiting 6

6.2 Användarroller (RBAC) för webbgränssnitt 6

7. API-dokumentation 6

8. Projektstruktur 7

9. Framtida förbättringar 7

1. Inledning

Detta projekt är en fullstack IoT-lösning för övervakning av miljösensordata, med fokus på strandvattenkvalitet. Systemet simulerar en IoT-enhet, samlar in data via ett REST-API, lagrar datan i en databas och visualiserar den i realtid via en webbaserad dashboard. Projektet är fullt förberett för produktion och kommunal överlämning genom användning av Docker, PostgreSQL, säkerhetsmekanismer och inbyggd övervakning.

2. Arkitektur

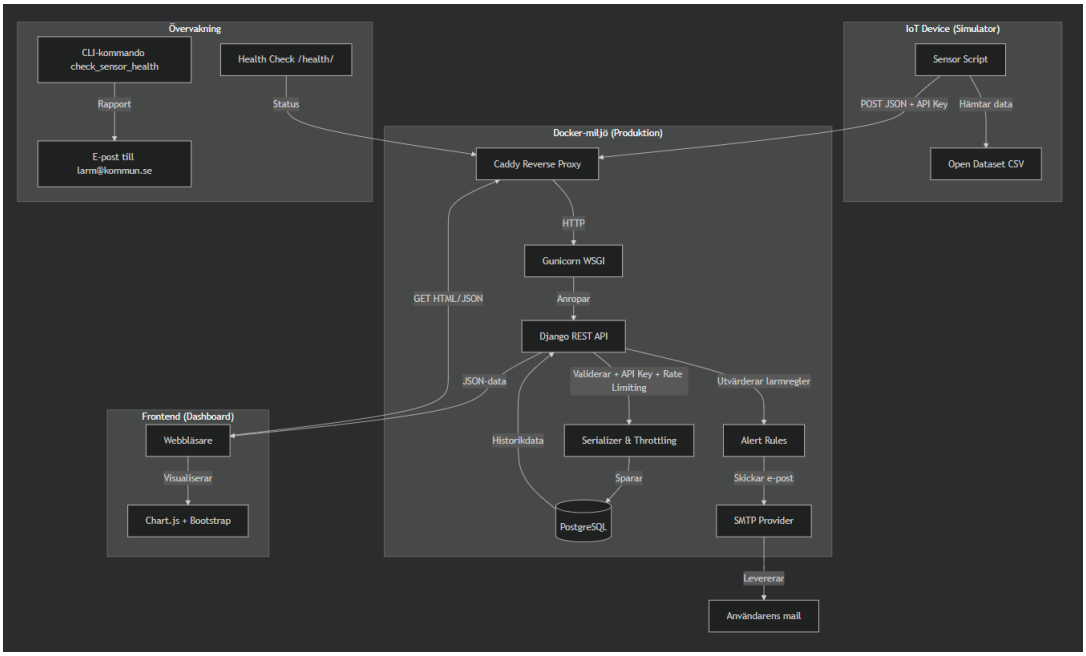
Systemet består av fyra huvudkomponenter, paketerade i en modern containerarkitektur:

1. **Sensor (simulator)** – En Python-applikation som simulerar en IoT-enhet. Den hämtar verklig data från ett öppet dataset, autentiserar sig med en API-nyckel och skickar datan till backend.
2. **Infrastruktur och nätverk** – En reverse proxy (Caddy) hanterar inkommande trafik och dirigerar den till applikationsservern (Gunicorn) som kör Django-applikationen.
3. **Backend (API och databas)** – Django tillsammans med Django REST Framework (DRF) tar emot, validerar och rate-limitar datan. Datan lagras persistent i en PostgreSQL-databas.
4. **Frontend (dashboard)** – Ett webbgränssnitt som visar data i realtid, historiska trender, aktuella larm samt övervakar sensorernas hälsa (monitoring).

Arkitekturflöde (textuell beskrivning)

- Simulatorn läser ett CSV-dataset och skickar JSON-data via HTTP POST till Caddy-proxyn.
- Caddy vidarebefordrar anropet till Gunicorn som kör Django.
- Django validerar API-nyckeln, tillämpar rate limiting och sparar datan i PostgreSQL.
- Vid överskridna tröskelvärden (t.ex. vattentemperatur över 20 grader) skickas e-post via SMTP.
- Webbdashboarden hämtar data via API-anrop och visualiserar med Chart.js.

Se figur 1 nedan för systemdataflöde diagram.



Figur 1: dataflödet systemdiagram

3. Teknikval

Backend: Django och Django REST Framework (Python)
Valdes för sitt robusta, säkra och skalbara ramverk. Inbyggt stöd för ORM, administratörsgränssnitt och REST-API. DRF erbjuder kraftfulla verktyg för throttling (rate limiting) och serialisering.

Databas: PostgreSQL
Standardval för relationsdatabaser i produktion. Hanterar stora datamängder och hög samtidighet bättre än SQLite. SQLite stöds dock fortfarande för snabb lokal utveckling utan Docker.

Infrastruktur: Docker, Gunicorn och Caddy

- Docker garanterar identisk körning oavsett miljö och förenklar distribution.
- Gunicorn är en robust WSGI-server anpassad för produktion.
- Caddy är en snabb reverse proxy som automatiskt kan hantera HTTPS/TLS.
- WhiteNoise används för effektiv servering av statiska filer direkt från Gunicorn.

Frontend: Bootstrap 5 och Chart.js

- Bootstrap för responsiv design.
- Chart.js för enkel men kraftfull datavisualisering.

4. Kravuppfyllelse

Se tabell 1 och 2 nedan för kravspecifikation:

Funktionella krav

Tabell 1: Funktionella krav

Krav	Status	Beskrivning
Simulerad sensor	Uppfyllt	Hämtar data, tidsstämplar, autentiserar sig och skickar till API.
Datakommunikation	Uppfyllt	Överför data via REST API (POST) säkert med API-nyckel.
Backend och API	Uppfyllt	Exponerar REST API, validerar input och tillämpar rate limiting.
Dataplattform	Uppfyllt	Lagrar sensordata persistent i PostgreSQL (eller SQLite lokalt).
Frontend	Uppfyllt	Visar mätvärden, diagram, sensorstatus och aktiva larm.
Ärendehantering	Uppfyllt	Formulär och flöde för inloggade användare och managers.
Larm och notifieringar	Uppfyllt	E-post skickas baserat på regler (exempelvis vattentemperatur> 20°C).
Övervakning (monitoring)	Uppfyllt	Health-endpoint, sensorhälsa i UI och driftlarm via CLI.

Icke-funktionella krav

Tabell 2: Icke-funktionella krav

Krav	Status	Beskrivning
Produktionsmiljö	Uppfyllt	Containeriserad med Docker, Gunicorn och Caddy. Miljövariabler används.
Säkerhet	Uppfyllt	Enhetsautentisering (API-nyckel) för IoT, rollbaserad åtkomst (RBAC) för webb, rate limiting för API.
Versionshantering	Uppfyllt	Git har använts genom hela projektet.

5. Instruktioner för körning

5.1 Produktion (Docker Compose) – rekommenderas

1. Kopiera filen .env.example till .env och sätt unika värden (till exempel DEVICE_API_KEY och SECRET_KEY).
2. Kör kommandot: docker compose up -d --build
   Detta startar PostgreSQL, bygger webbcontainern, kör migreringar (migrate), samlar in statiska filer (collectstatic) och startar Caddy-proxyn.
3. Tjänsten nås via http://localhost:18080 (standardporten som anges i .env.example).
4. Starta simulatorn: python sensor/simulator.py
   Simulatorn läser automatiskt konfiguration från .env, inklusive BACKEND_URL och DEVICE_API_KEY.
   Vid Docker-körning ska BACKEND_URL peka på Caddy-proxyn: http://localhost:18080/api/data/.
   Simulatorn skickar sensordata för alla stränder i datasetet och sorterar datan kronologiskt (baserat på mättidsstämpeln) för att simulera ett realistiskt flöde.
5. Dashboarden nås via http://localhost:18080/dashboard/.

5.2 Lokal utveckling (utan Docker)

1. Aktivera en virtuell miljö (venv) och kör pip install -r requirements.txt.
2. Kör python manage.py migrate för att sätta upp SQLite-databasen.
3. Starta servern med python manage.py runserver.
4. Starta simulatorn med rätt DEVICE_API_KEY i miljön.

5.3 Övervakning och driftlarm

Systemet övervakar kontinuerligt om sensorer slutar skicka data (blir så kallade ”stale”).

- **Health check:** GET /health/ returnerar status 200 OK och en JSON som bekräftar att webb och databas fungerar, till exempel {"status":"ok","database":"connected"}.
- **Driftlarm (CLI):** Kör docker compose exec web python manage.py check_sensor_health --stale-minutes 30 --email-to larm@kommun.se för att få en rapport om inaktiva sensorer. Detta kommando är lämpligt att köra som ett Cron-jobb.

6. Säkerhet och åtkomstkontroll

6.1 Enhetsautentisering (IoT) och rate limiting

För att förhindra skräpdata och överbelastning har REST-API:et skyddats på två sätt:

1. **API-nyckel** – Anrop till POST /api/data/ kräver en hemlig nyckel. Nyckeln kan skickas antingen via headern X-Device-Key eller som en Authorization: Bearer <nyckel>. En ogiltig nyckel ger svaret 401 Unauthorized.
2. **Rate limiting (throttling)** – Djangos REST Framework begränsar antalet anrop:
    - sensor_ingest: max 120 anrop per minut (skyddar mot spam av mätdata).
    - sensor_read: max 600 anrop per minut (skyddar API-läsningar från dashboard eller tredjepart).

6.2 Användarroller (RBAC) för webbgränssnitt

Se tabell 3 för användargränssnitt

Tabell 3: Användargränssnitt

Roll	Beskrivning	Behörighet
Admin	Systemadministratör	Fullständig tillgång (Django Admin + alla vyer).
Manager	Beslutsfattare	Tillgång till rapportvy och hantering av ärenden.
Analyst	Dataanalytiker	Tillgång till analysvy.
Viewer	Observatör	Endast läsrättigheter i dashboarden.
Roll	Beskrivning	Behörighet

7. API-dokumentation

Se tabell 4 för api

Tabell 4: API

Metod	Endpoint	Beskrivning	Autentiseringskrav
GET	/health/	Health check för övervakning av webb och databas.	Ingen
POST	/api/data/	Tar emot nya sensorvärden från simulatorn.	API-nyckel (X-Device-Key)
GET	/api/data/latest/	Hämtar senaste värdet. Queryparametrar: sensor_id	Inloggad användare
GET	/api/data/history/	Hämtar historik. Queryparametrar: sensor_id, from, to	Inloggad användare
GET	/api/incidents/	Exporterar ärenden som JSON.	Manager eller Admin

8. Projektstruktur

Mappen smart-env-monitor/ innehåller följande undermappar och filer:

- api/ – REST-API (DRF), autentisering, throttling och larm.
    - management/ – Egna CLI-kommandon (till exempel check_sensor_health).
- backend/ – Django-projektkonfiguration.
- dashboard/ – Webbdashboard (templates och vyer).
- sensor/ – IoT-simulatorn.
- docker-compose.yml – Konfiguration för Docker-stacken (databas, webb, proxy).
- Dockerfile – Instruktioner för att bygga webbcontainern.
- Caddyfile – Konfiguration för reverse proxy Caddy.
- requirements.txt – Lista över Python-beroenden.
- doc/ – Dokumentation och bilagor.

9. Framtida förbättringar

Eftersom plattformen nu är dockeriserad, säkrad och utrustad med en produktionsdatabas (PostgreSQL), är de huvudsakliga framtida stegen:

1. **Realtid** – Implementera WebSockets (Django Channels) för att pusha data till dashboarden utan att behöva polla.
2. **Fysisk hårdvara** – Ersätta simulatorn med exempelvis en Raspberry Pi eller ESP32 och riktiga sensorer ute i fält.
3. **E-post i produktion** – Verifiera egen domän (SPF/DKIM/DMARC) och skicka från noreply@egen-domän.se för bättre leveransbarhet.
4. **Bakgrundsjobb** – Bryta ut e-postutskick och regelutvärdering till en asynkron worker (till exempel Celery med Redis) för att säkerställa minimal svarstid i API:et vid hög belastning.